

Project Overview: Regional Tasting Shuttle Platform

Hello! We are looking for a Lead Engineer to partner with us on an upcoming web application build. We have already completed the initial architecture research and technical planning. Before we officially kick off, we want you to review the scope, constraints, and chosen stack to ensure you are aligned with our approach and comfortable taking on this project as a contractor.

The Goal: We are building a lightweight, highly efficient tracking and logistics platform for a regional tasting shuttle service. We are optimizing for a small footprint, rapid delivery, and extremely low maintenance. The principal engineer has already locked in the core tech stack (Supabase, React+Vite, MapLibre), which is detailed in the brief below.

Your Role: Review the architecture and constraints. If you agree to take this on, your first phase of work will involve tackling a specific list of "research spikes" (detailed at the end of this document) to pressure-test a few key technical decisions before we transition into full production mode.

Architecture & Stack Brief

Audience: Lead Engineer (build research + technical planning)

From: Surya · Agentic Architects

Status: Pre-build. This brief is your starting point to research build specifics and pressure-test choices.

1. Mission in one paragraph

Build a single web app, hosted in the cloud, that does three things: (1) lets anonymous riders scan a QR code, enter a daily access code, and watch the shuttle move on a live map with next-stop ETAs; (2) gives drivers a dead-simple mounted-tablet screen to log stop arrivals and passenger on/off counts; (3) gives the owner an admin dashboard to generate the daily code, watch buses live, assign buses to routes, and read a handful of business-health metrics. We are launching with two buses on one shared primary loop, expanding to ~4 buses and a secondary route in 6–12 months.

2. Hard constraints (read these first)

Constraint	Detail	Implication
Strict Scope	Fixed-scope build.	Our discipline is key. Every avoidable component adds overhead. Minimize moving parts.
Timeline	Client operations launch this weekend (May 23/24), but buses will run manual drop-offs for 1–2 months. The tracking app is mandatory in 4–6 weeks.	The real deadline is ~mid-June to mid-July. The timeline is comfortable.
No Accounts for Riders	Access is a daily rotating code, valid ~10am–6pm. No auth, no booking integration, no payments.	Removes the entire identity/booking layer. Riders are strictly anonymous.

3. Guiding principles (non-negotiable design stance)

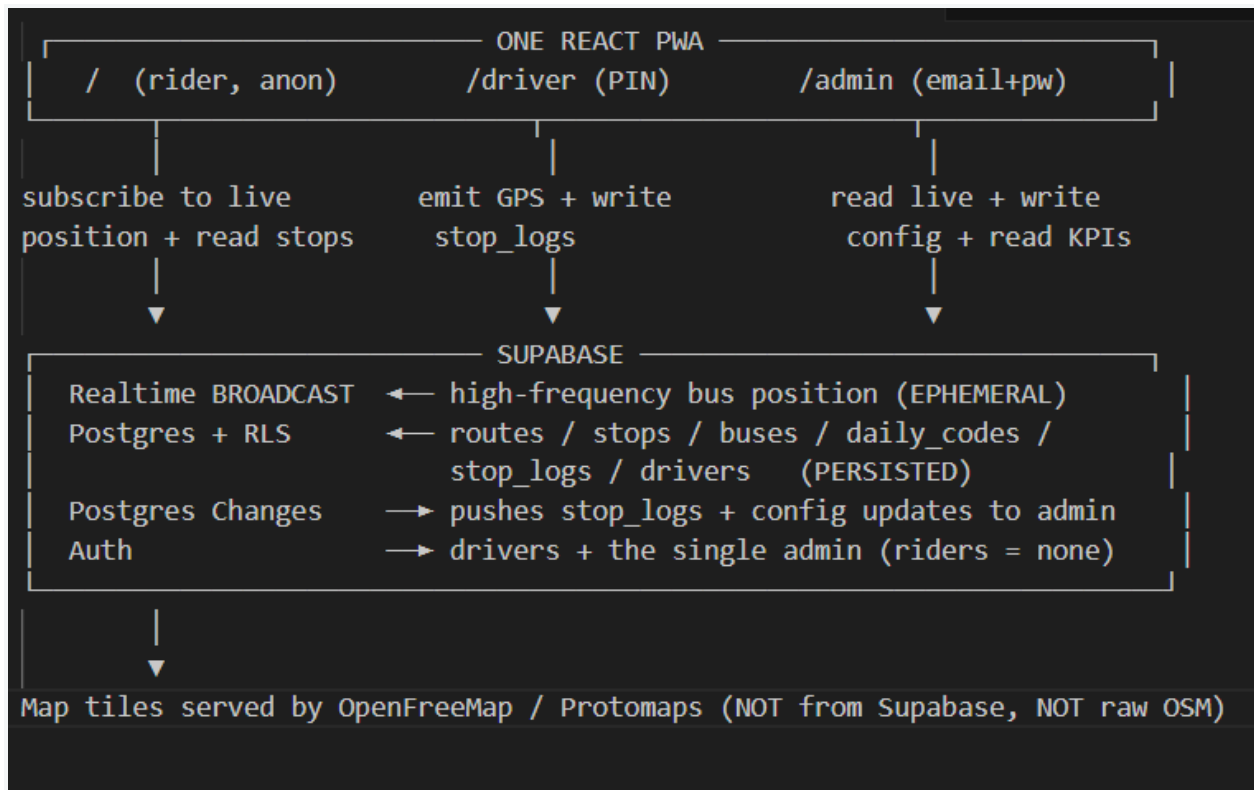
- **Minimum components:** Fewer moving parts = faster delivery, lower hosting complexity, less to maintain. We collapse "backend + realtime + auth + DB" into one managed service.
- **Off-the-shelf, open-source, boring:** Prefer mature OSS we configure over anything we hand-build. If a boring tool ships faster, it wins.
- **Web-only:** No native iOS/Android. One codebase, three role-based views. No app-store review (a delivery killer), no dual-platform overhead.
- **Flow over stock:** The data model records independent per-stop events (boarded/alighted), never a stateful running occupancy. One missed tap breaks one row, not the whole day. (See §6.)
- **Config, not code:** Routes/stops/buses are data with foreign keys. Scaling from 2→4 buses or adding the secondary route is admin data entry, never a code change or redeployment.

4. The stack

Layer	Choice	Role	Why this
Backend-as-a-service	Supabase (OSS, self-hostable)	Postgres DB + Realtime (WebSockets) + Auth + Row-Level Security	Collapses ~4 components into one managed box. Almost no bespoke server. RLS enforces roles at the DB. Near-zero ops.
Frontend	React + Vite	Single PWA, role-based routes (/ , /driver, /admin)	One codebase; installable to home screen; no app store needed.
Map renderer	MapLibre GL JS	Draws the interactive map	Free, no per-load billing, bundled via npm.
Map tiles	OpenFreeMap (default) or self-hosted Protomaps .pmtiles	Supplies the actual map imagery/vector data	 Do NOT use raw tile.openstreetmap.org its usage policy forbids production embedding. This is the #1 mistake to avoid.
Frontend hosting	Vercel or Netlify	Serves the PWA	Git-push deploys, near-zero maintenance.
Driver tablet runtime	Fully Kiosk Browser (Android)	Locks the tablet to /driver, holds screen awake	Turns a cheap Android tablet into a reliable mounted

Layer	Choice	Role	Why this
			appliance.

5. System architecture & data flow



The most important transport decision: position vs. records

Industry pattern for driver-location dashboards: split the two data types.

1. **Live bus position (every 3–5s):** Send via Supabase Realtime Broadcast — ephemeral, no database write per ping. Riders/admin subscribe to a per-route channel and see movement live. We do not persist every GPS ping (avoids write amplification, keeps the DB tiny, dodges Postgres-Changes WAL latency under load).
2. **Stop events (arrivals, on/off counts):** These are facts → write to the stop_logs Postgres table. Admin dashboard listens via Postgres Changes for live updates. These are low-frequency (a handful per loop), so latency and volume are non-issues.

3. *Optional:* Persist a last-known-position snapshot per bus (one upsert row) so a freshly-opened map isn't blank before the next broadcast. Decide based on "do you need history?" answer.

6. Data model (the entire core)

```
routes (  
  id, name, active          -- "Primary loop", "Secondary loop"  
)  
  
stops (  
  id, route_id FK, name, sequence,  -- ordered stop list per route  
  scheduled_time, lat, lng          -- scheduled_time drives the on-time KPI + ETA  
)  
  
buses (  
  id, label, route_id FK, status    -- admin flips route_id (config-not-code);  
                                     -- status = in_service | off_service | offline  
)  
  
daily_codes (  
  id, code, valid_date, route_id FK -- one row/day  
)  
  
drivers (  
  id, name, pin_or_auth_ref, active -- write-only role via RLS  
)  
  
stop_logs (  
                                     -- ★ APPEND-ONLY. The single source of truth.  
  id, bus_id FK, stop_id FK, route_id FK,  
  service_date,                    -- date column → trivial daily GROUP BY, no TZ math  
  arrived_at,                      -- timestamp → on-time KPI + time-of-day buckets  
  boarded DEFAULT 0,               -- FLOW event, not a running total  
  alighted DEFAULT 0  
)
```

Why flow-over-stock matters operationally: The driver taps "Arrived" at every stop (creating the row, defaulting counts to 0), increments on/off only based on real movement, then taps

"Confirm" to lock the row. Result: "visited, nobody moved" becomes a trustworthy data point. Every dashboard metric is a single GROUP BY over this one table. Live occupancy ("fullness") is the only stock-style number and is treated as an approximate gauge that resets each loop, never a reported metric.

7. Auth & roles (Row-Level Security does the heavy lifting)

Role	Auth	Can do	RLS posture
Rider	None (anonymous). Daily-code check is an RPC.	Read in-service bus positions + stops + offers for a route (after code validation).	No table access beyond what the public RPC/views expose.
Driver	Lightweight login (name + 4-digit PIN).	Insert/update their own bus's stop_logs; set their bus status; emit position.	Write-only to own bus; cannot read admin data or other buses' detail.
Owner/Admin	Email + password (single account).	Everything: generate codes, CRUD routes/stops, assign drivers, read logs/KPIs.	Full access.

8. Maps, tiles & ETA

- **Renderer:** MapLibre GL JS, npm-bundled (no third-party script tracking loads).
- **Tiles:** See §4. Default to OpenFreeMap; evaluate self-hosted Protomaps for lowest long-term risk/control.
- **ETA:** For V1, compute ETA from the schedule (next stop scheduled_time – now, adjusted by observed delay) rather than standing up a routing engine. A full routing/road-network ETA is out of scope.

9. The driver-tablet reality (the one physical gotcha)

Browser geolocation (watchPosition) is reliable only while the page is foregrounded and the screen is on. We neutralize this by making tablet provisioning a core deliverable:

- **Screen timeout:** Never. Tablet is mounted + charging in the bus.
- Hold the **Screen Wake Lock API** while /driver is open.
- Lock the device into **Fully Kiosk Browser**, pinned to the /driver URL.
- We ship the tablet pre-configured. (An un-provisioned tablet that sleeps = a bus that vanishes from the map = a support call that's our fault).

10. Offline strategy (deliberately minimal for V1)

- **V1 (Primary route, has coverage):** Light buffer-and-flush. Queue GPS pings and stop taps in IndexedDB when the signal blips, replay in order to reconnect so the app never crashes or loses a count. Cheap insurance.
- **Phase 3 (Secondary route, real dead zones):** Robust offline sync. This is a separate future phase. Do not build it now.

11. Scalability story

Everything hangs off route_id. Adding buses or the secondary route is just rows in buses/routes/stops, surfaced through the admin UI as dropdowns and forms. No schema change, no redeploy. Build it right once on day one.

12. Research spikes for you (the actionable list)

Please come back with findings + a recommendation on each:

1. **Tile provider decision:** OpenFreeMap vs self-hosted Protomaps .pmtiles vs MapTiler. *Deliverable: a recommendation + a working MapLibre map of the target region on the chosen tiles.*
2. **Supabase Realtime Broadcast for position:** Confirm the Broadcast (ephemeral) + Postgres-Changes (stop_logs) split holds for 2–4 buses pinging every 3–5s, including reconnection behavior. *Deliverable: a tiny spike — one tab emits a moving marker, another subscribes.*
3. **Driver auth pattern:** Decide the cleanest mapping for "name + 4-digit PIN" within Supabase Auth (e.g., per-driver email alias + PIN-as-password, or anonymous sign-in + custom claim). *Deliverable: recommended pattern + RLS policy sketch.*
4. **Wake Lock + Fully Kiosk reliability:** Test on a specific budget Android tablet across an 8-hour shift (screen never sleeps, GPS keeps emitting). *Deliverable: go/no-go + the exact kiosk config recipe we'll ship.*
5. **IndexedDB buffer-flush:** For GPS + tap queue with in-order replay. *Deliverable: the queue/replay approach.*
6. **Schedule-based ETA math:** Run against a sample loop timetable. *Deliverable: the formula + edge cases (bus ahead of schedule, skipped stop).*
7. **PWA install + permissions UX:** Geolocation prompt on the driver tablet; "add to home screen" for riders. *Deliverable: the permission/onboarding flow.*

14. Open technical risks

- Tile licensing/cost drift if we pick a SaaS provider and volume grows: Protomaps self-host de-risks this.
- Web geolocation on the tablet is the highest-risk dependency; spike #4 must clear before we commit to the architecture.
- Driver compliance with the mandatory "Arrived" tap is an operational risk that directly degrades data quality; the UI must make it the path of least resistance.

15. Suggested shape

- **Monorepo**, single React+Vite PWA, route-based code-splitting per role.
- **Supabase project** with migrations in-repo (supabase/migrations), RLS policies version-controlled.
- **Two environments:** dev → prod. Vercel/Netlify preview deploys per PR.